# Stateful Swarms for Document Intelligence

Auditability, Cost Efficiency, and Persistent Learning in Multi-Agent Systems

**Devansh**

Irys AI (Iqidis, Inc.)

devansh@iqidis.ai

June 2026

---

**Repository:** github.com/dl1683/irys-stateful-swarms
**License:** MIT — use however you want, including commercially.
**Results:** 83.74% criteria pass on 1,251-task Harvey Legal Agent Benchmark at $1.30/task.

---

**Abstract**

We present a swarm-based architecture for long-context document analysis that coordinates multiple AI agents through a shared, evolving blackboard. Unlike monolithic single-prompt approaches, the system decomposes tasks into planning, extraction, analysis, and synthesis phases — each producing structured, source-grounded state that can be inspected, debugged, and extended. We evaluate on the full 1,251-task Harvey Legal Agent Benchmark (LAB), achieving 83.74% pooled criteria pass rate and 17.75% strict all-pass at $1.30 per task. We argue that the primary value of swarm coordination is not raw performance but three properties that monolithic systems cannot provide: full auditability of the reasoning process, predictable cost scaling, and the foundation for persistent cross-session learning that reduces marginal cost over time. We open-source the complete system, all benchmark outputs, and five annotated reasoning traces under the MIT license.

# 1 Introduction

The dominant approach to applying large language models to professional document work is the single-pass pipeline: ingest documents, construct a prompt, call a model, return the output. This approach has three structural problems that become severe as task complexity increases.

**Opacity.** A single model call produces an answer but no explanation of how that answer was derived. When the output is wrong — a missed clause, an incorrect calculation, a misattributed party — there is no intermediate state to inspect. The failure is visible only in the output, not in the reasoning that produced it. For professional work where accountability matters, this is not a minor limitation.

**Cost unpredictability.** Single-pass systems send the entire document context to the most capable (and most expensive) model for every query. A simple factual extraction and a complex multi-document comparison cost the same. There is no mechanism for routing simpler subtasks to cheaper models or for avoiding redundant processing of previously analyzed material.

**Statelessness.** Each invocation starts from zero. A system that analyzed a 200-page credit agreement yesterday has no memory of that analysis today. Every follow-up question, every new document added to the same matter, triggers a full re-processing cycle. The cost of the tenth question is identical to the cost of the first.

We propose that multi-agent swarm coordination with structured shared state addresses all three problems simultaneously. The key insight is that the intermediate state — the blackboard — is not just an implementation detail. It is the primary artifact of the system, more valuable than the final output it produces.

# 2 Architecture: The Blackboard Pattern

## 2.1 Core Design

The system coordinates multiple AI agents through a shared blackboard — a structured, append-only knowledge base that evolves over multiple iterations. Each agent reads from the blackboard, performs a specific type of cognitive work, and writes its findings back as typed entries with source provenance.

The pipeline proceeds through ordered phases:

1. **Seed Planning.** Before any document is read in detail, a planning agent scans document structure (headings, tables, estimated complexity) and produces: key questions the investigation must answer, extraction focus areas per document, an analytical framework, completeness criteria, and a task-specific state map defining what object types and fields must be populated.

2. **Parallel Extraction.** Worker agents read source documents and write structured findings to the blackboard. Each finding carries a type (observation, analysis, calculation, strategy, gap, contradic-

tion), source provenance (document name, section, evidence quote), confidence, and epistemic classification (fact, inference, expert opinion, adversarial claim).

3. **Analytical Review.** A stronger model reviews accumulated state, identifies cross-document patterns, resolves contradictions, and produces higher-order analytical entries that reference the evidence entries they synthesize.
4. **Convergence Check.** A supervisor assesses whether the blackboard is complete enough to answer the task. If not, it dispatches targeted follow-up workers with specific questions derived from identified gaps.
5. **Obligation Extraction.** The system converts blackboard state into concrete deliverable obligations — specific items that must appear in the final output, each traced back to source entries.
6. **Curation.** The most relevant entries are selected and ordered for synthesis, enforcing minimum density thresholds to prevent thin outputs.
7. **Synthesis.** The final deliverable is produced from curated state. Large tasks use sectioned synthesis, drafting each section independently to avoid output truncation.
8. **Verification.** Deterministic and model-based checks confirm the output meets structural requirements. Missing obligations trigger targeted augmentation.

### 2.2 Model Routing

Not every phase requires the same model capability. The system routes work to models based on the cognitive demands of each phase:

| Phase | Model Tier | Rationale |
|---|---|---|
| Extraction | Lightweight | High-volume, factual, parallelizable |
| Planning, Analysis, Synthesis | Mid-tier | Requires reasoning, cross-referencing |
| Convergence, Verification | Mid-tier | Quality judgment |

This routing is the primary mechanism for cost control. Extraction — which constitutes the majority of model calls — uses the cheapest available model. Only phases requiring genuine reasoning use more capable (and expensive) models.

### 2.3 Signal Lifecycle

The blackboard maintains a signal queue: typed questions, read requests, and investigation prompts that guide worker behavior. Signals are generated during seed planning and dynamically during extraction as gaps are discovered. Each signal tracks its status (open, addressed, expired) and which entries address it. This creates a self-correcting feedback loop: the system generates questions, workers attempt to answer them, and the convergence check evaluates whether the answers are sufficient.

## 3 Evaluation

### 3.1 Benchmark

We evaluate on the Harvey Legal Agent Benchmark (LAB), a public benchmark consisting of 1,251 tasks across 24 legal practice areas. Each task provides source documents (contracts, filings, correspondence, spreadsheets) and an instruction requiring the system to produce one or more deliverables (memos, analyses, redlines, spreadsheets). Tasks are scored against per-criterion rubrics; strict all-pass requires every criterion to be satisfied.

Harvey's published results on their private holdout set (which mirrors the public distribution) report 10.4% strict all-pass at approximately $50.90 per task. We do not have access to the private holdout and would welcome the opportunity to run on it for a direct comparison.

### 3.2 Results

| Metric | Result |
|---|---|
| Tasks completed | 1,251 / 1,251 |
| Criteria pass rate | 62,800 / 74,990 = 83.74% |
| Strict all-pass | 222 / 1,251 = 17.75% |
| Success gate ($\leq$2 misses or $\geq$95%) | 336 / 1,251 = 26.86% |
| Total cost | $1,626.08 |
| Cost per task | $1.30 |

Performance varies significantly by task type:

| Task Type | Tasks | Criteria % |
|---|---|---|
| draft | 427 | 90.22 |
| analyze | 89 | 86.42 |
| scenario | 119 | 83.91 |
| review | 58 | 80.86 |
| compare | 148 | 77.14 |
| extract | 138 | 75.67 |
| identify | 195 | 74.26 |

The system is strongest on generative tasks (drafting, analysis) where partial state can still produce useful output, and weakest on exhaustive recall tasks (extraction, identification) that require complete enumeration of specific facts. This asymmetry is informative: it reveals that the primary bottleneck is state completeness, not synthesis quality.

### 3.3 Architecture Over Model Intelligence

These results deserve scrutiny because of the models that produced them. The system defaults to **Gemini 3.1 Flash Lite** ($0.25/M input tokens) for extraction workers and **Gemini 3.5 Flash** ($1.50/M input tokens) for planning, analysis, and synthesis. These are among the cheapest models available from any provider.

Harvey's published LAB results include per-model breakdowns across multiple agentic systems. Gemini models — the same model family used here — achieved **0% strict all-pass** across Harvey's agentic evaluations. The same models that produce zero successful tasks in other agentic architectures achieve 17.75% strict all-pass when coordinated through a stateful swarm.

This gap between individual model capability and coordinated system performance is the central empirical finding of this work. It demonstrates that the performance comes from the architecture — swarm coordination, structured state-building, typed provenance tracking, signal-driven gap identification, and multi-iteration convergence — not from model intelligence. Expensive frontier models are not required. The engineering of how models are coordinated and how their outputs are structured, accumulated, and reconciled matters more than the capability of any individual model.

We deliberately open-source with cheap models as the default to make this point concrete and to make the system genuinely accessible. Anyone with a Gemini API key can run a complete professional document analysis task for $1.30.

### 3.4 Cost Analysis

The $1.30 per task average represents a 39x reduction compared to Harvey's published $50.90 figure. This reduction comes from three sources:

1. **Model routing.** The majority of extraction work uses lightweight models at $0.25/M input tokens, while only planning, analysis, and synthesis use mid-tier models at $1.50/M input tokens.
2. **Parallel extraction.** Workers process document sections concurrently, reducing wall-clock time without increasing per-token costs.
3. **Targeted follow-up.** Rather than re-processing entire documents when information is missing, the convergence check dispatches focused workers to specific sections, avoiding redundant processing.

## 4 Auditability

### 4.1 The Case for Structured Reasoning Traces

The most significant property of the blackboard architecture is not performance but auditability. Every task produces a complete, structured record of how the system arrived at its conclusions.

Each blackboard snapshot is a JSON document containing every entry the system has produced, with full provenance:

```
{
  "id": "e716",
  "type": "observation",
  "content": "Northbrook Capital Markets, LLC commits to provide
    a first lien senior secured term loan B facility in an
    aggregate principal amount of $350,000,000.",
  "source": {
    "document": "commitment-letter.docx",
    "section": "Full Document (part 1)"
  },
  "created_by": {
    "worker_id": "reader_commitment-letter.do",
    "description": "initial_reading",
    "iteration": 0
  },
  "confidence": 0.9
}
```

This entry records not just *what* was found, but *where* it was found, *who* found it (which worker), *when* in the process it was found (iteration 0), and *how confident* the system is in the finding.

### 4.2 Auditing Perfect Scores

When the system succeeds, the reasoning trace demonstrates *why* it succeeded. In a credit agreement comparison task that scored 40/40, the blackboard evolved from 7 seed planning entries to 2,400 grounded findings over 12 iterations. The system:

- Generated 12 targeted questions before reading any documents
- Extracted 2,044 source-grounded observations with exact dollar amounts, dates, and clause references

- Produced 87 calculations cross-referencing terms across documents
- Identified 135 gaps requiring follow-up extraction
- Built 113 cross-document analysis entries identifying specific deviations

The final analysis entries contain findings such as:

```
{
  "id": "e865",
  "type": "analysis",
  "content": "Section 2.06 of the draft credit agreement specifies
    an annual agency fee of $50,000. This is a deviation from the
    Commitment Letter, which requires an Administrative Agent Fee
    of $150,000 per annum, payable annually in advance.",
  "created_by": {
    "worker_id": "flash35_analyst",
    "description": "direct_analysis",
    "iteration": 12
  },
  "confidence": 0.98,
  "supports": ["e260", "e261", "e35"]
}
```

The `supports` field creates an explicit evidence chain: this analysis entry is grounded in three earlier observations, each of which carries its own source provenance. A reviewer can follow the chain from conclusion to evidence to source document.

### 4.3 Auditing Failures

More importantly, when the system fails, the reasoning trace reveals *why* it failed — and whether the failure is fundamental or fixable.

In a sanctions entity extraction task (80/85), the system missed five criteria. Examining the blackboard reveals that in every case, the relevant information was present in the state but the final verification step was incomplete:

**Missed criterion: "Beneficiary name inconsistency."** The blackboard contained two entries with different name variants — "Zenith Petrochem Industries LLC" in one entry and "Zenith Petrochemical Industries LLC" in another. Both name variants existed in the state. The system extracted the facts correctly from both documents. What it failed to do was cross-reference these entries and flag the discrepancy. The information was there; the cross-reference step wasn't.

**Missed criterion: "OFAC 50% rule aggregation principle."** The system identified the threshold proximity (49% vs 50%) and even mentioned that aggregate ownership by blocked persons could trigger a violation — but didn't elaborate on the aggregation principle with sufficient specificity for the scorer.

This pattern — correct extraction, incomplete verification — is qualitatively different from a system that never found the relevant information. It indicates that the failure mode is in the state processing pipeline, not in the extraction capability. These are fixable failures, and the blackboard makes it possible to identify them precisely.

### 4.4 Implications for Professional Adoption

For legal work specifically, auditability is not optional. Attorneys are professionally responsible for the accuracy of their work product. A system that produces correct answers 83% of the time is useful only if

the remaining 17% can be identified and corrected. The blackboard pattern provides the mechanism for this: every conclusion is traceable to evidence, every evidence entry is traceable to a source document, and every gap is explicitly logged.

This is fundamentally different from a system that produces a final document with no intermediate reasoning. Even if the final document is correct, the lack of a reasoning trace means there is no way to verify *why* it is correct — and no way to determine whether a specific conclusion is well-supported or hallucinated.

# 5 The Case for Persistent Stateful Swarms

## 5.1 The Benchmark Constraint

The results presented above were produced under a significant constraint: every task starts from an empty blackboard. The system has no memory of previous tasks, no accumulated domain knowledge, and no persistent document indexes. This is the correct approach for benchmark evaluation — persistent state would constitute an unfair advantage.

However, this constraint means the benchmark results systematically understate the performance and cost efficiency of a deployed system.

## 5.2 What Persistence Enables

In a production deployment, the system operates within the context of ongoing matters — deals, cases, investigations, compliance programs. These matters involve recurring documents analyzed repeatedly as new questions arise, accumulated understanding from early analysis that remains valid for subsequent queries, and incremental updates where new documents must be reconciled against existing state.

A persistent swarm would retain its blackboard state across sessions. The 2,400 entries produced during the credit agreement analysis would persist. When a follow-up question arrives, the system would query existing state rather than re-reading the entire document set. When a new amendment arrives, the system would reconcile it against the existing entity graph and flag changes.

## 5.3 Deterministic Operations on Typed State

A critical property of the blackboard is that much of the analytical work performed on persistent state does not require model invocations at all. The typed, provenance-tracked entry structure enables deterministic algorithms that operate directly on the state graph:

**Confidence propagation.** When a new entry supports or contradicts an existing entry, confidence scores update deterministically based on the relationship type, source diversity, and corroboration count. No model call is needed — the provenance metadata carries enough structure for arithmetic updates.

**Contradiction detection.** When entries with conflicting content are added, the system automatically creates contradiction entries, opens critical-priority signals, penalizes confidence on both sides, and propagates the penalty to downstream entries that depend on the conflicted ones. This entire cascade is deterministic.

**Supersession tracking.** When new information supersedes an older entry, the superseded entry is marked inactive and any signals it had addressed are reopened for re-evaluation. The system maintains a complete version history without model involvement.

**Entity resolution and obligation tracking.** Cross-referencing entities across entries, tracking which obligations have been satisfied, and identifying gaps in coverage are all operations on typed structured data — graph traversals and set operations, not natural language tasks.

The implication for cost is significant: in a stateful deployment, the majority of follow-up work — querying existing state, resolving entities, checking obligation coverage, detecting conflicts with new information — never touches an LLM. Only genuinely new cognitive work (reading a new document, performing novel cross-document analysis, synthesizing a new deliverable) requires model calls.

### 5.4 Cost Implications

The cost structure of the open-source benchmark system is dominated by extraction — lightweight worker calls constituting roughly 70% of total model spend. With persistent state, subsequent queries on the same document set skip extraction entirely and proceed directly to analysis and synthesis from cached state.

Irys, our production platform, takes this further. By combining stateful swarm coordination with hierarchical embeddings, persistent knowledge graphs, entity linking, and the deterministic state operations described above, Irys reduces the cost of multi-turn inference by up to **1,000x** compared to stateless re-computation. The system does not spend tokens constantly re-reading documents, re-extracting entities, or re-deriving analyses it has already performed. Typed provenance tracking allows the system to deterministically isolate exactly which state needs updating when new information arrives — rather than re-processing everything, it targets only the affected subgraph. Combined with deterministic algorithms for entity resolution, obligation tracking, and conflict detection, the vast majority of follow-up work never touches an LLM at all.

For practices that work on recurring document types (credit agreements, compliance filings, standard contracts), the amortized cost per insight drops by orders of magnitude over time. This changes the economic model of AI-assisted document analysis from "pay full price for every question" to "invest in understanding a document set once, then query cheaply forever."

### 5.5 Learning Across Sessions

Beyond caching, persistent state enables a more sophisticated form of learning. As the system processes more documents within a domain, it accumulates entity knowledge (named entities, roles, relationships, prior appearances), domain patterns (common structures, typical formulations, standard obligation types), and quality feedback (which extraction strategies produced useful state).

This is not fine-tuning. The model weights remain unchanged. The learning happens in the structured state layer — the system becomes more effective because it has more context, not because it has been retrained. This preserves the auditability property: every piece of accumulated knowledge is traceable to the document that produced it.

### 5.6 Evaluation Isolation

Persistent state creates a tension with benchmark evaluation. If the system retains knowledge from previous benchmark tasks, its performance is no longer a clean measurement. This is why we deliberately strip all persistence for benchmark runs.

The solution is evaluation-aware persistence: strict boundaries between benchmark and production state, separate storage contexts, and audit mechanisms that verify no benchmark-derived knowledge contaminates generation. We have implemented a provenance audit system that monitors every model prompt for forbidden benchmark metadata and logs violations.

## 6 Related Approaches

The swarm architecture intersects with several active research directions that we believe are complementary:

**Latent space reasoning.** Inference-time perturbation of model activations can improve reasoning without training. We have demonstrated +19.6pp arithmetic improvement on frozen models through soft token injection. Applied to swarm worker models, this could improve extraction accuracy on the precise numerical and citation tasks where the current system is weakest.[1]

**Hierarchical embeddings.** Standard dense embeddings treat all dimensions equally, but document semantics are hierarchical. Multi-scale embeddings that align dimensional structure to semantic hierarchy improve retrieval quality. We have demonstrated that correct geometric hierarchy causally improves embedding quality.[2]

**Representation quality theory.** We have derived a universal law governing learned representation quality from first principles using extreme value theory, validated across 12 NLP architectures ($R^2$=0.955) and biological neural systems. This provides a theoretical framework for optimal model routing in multi-model swarms.[3]

**Persistent agent memory.** Provenance-backed knowledge memory systems that maintain source attribution, conflict state, and temporal validity across sessions are a necessary complement to the swarm pattern for production deployment.[4]

## 7 Limitations

**Recall on exhaustive tasks.** The system achieves 90%+ on generative tasks but 74–77% on extraction and identification tasks requiring complete enumeration.

**Single-run benchmark.** The results represent a single full benchmark run. We have not performed statistical analysis of run-to-run variance.

**Public benchmark only.** We do not have access to Harvey's private holdout set. We would welcome the opportunity to run on it for a direct comparison.

**No persistent state evaluation.** The cost and accuracy benefits of persistent state are projections based on observed cost structure, not measured results from a deployed persistent system.

**API-only.** The system uses only standard API calls to frontier language models. No fine-tuning, custom embeddings, or model modifications. This is both a limitation and a deliberate design choice — it establishes a baseline for what coordination alone can achieve.

## 8 Conclusion

We have presented a swarm-based document analysis architecture that achieves competitive benchmark performance at dramatically lower cost than published alternatives. The system's primary contribution is not raw performance but three structural properties: full auditability of the reasoning process through structured blackboard state, predictable cost through model routing and targeted processing, and a natural foundation for persistent cross-session learning.

The complete system is open-source under the MIT license. The repository includes source code, the full benchmark manifest, five annotated example tasks with complete reasoning traces, and verification instructions. Benchmark outputs for all 1,251 tasks are available for independent scoring.

We believe that stateful multi-agent coordination represents an important direction for professional AI systems — not because agents are inherently better than monolithic models, but because the struc-

[1] https://github.com/dl1683/Latent-Space-Reasoning
[2] https://github.com/dl1683/moonshot-fractal-embeddings
[3] https://github.com/dl1683/moonshot-cti-universal-law
[4] https://github.com/dl1683/MapU

tured intermediate state they produce is essential for accountability, cost control, and continuous improvement. We invite collaborators interested in extending this work to additional benchmarks, new problem domains, or production deployment.

**Contact:** devansh@iqidis.ai
**Repository:** github.com/dl1683/irys-stateful-swarms


## References

1. Harvey AI. "Legal Agent Benchmark Initial Results." https://www.harvey.ai/blog/legal-agent-benchmar 2025.
2. Harvey AI. "Opus 4.8 Now Live in Harvey." https://www.harvey.ai/blog/opus-4-8-now-live-in-harve 2026.
3. Harvey AI. Harvey Legal Agent Benchmark. https://github.com/harveyai/harvey-labs, 2025.